

**অপারেটিং সিস্টেমের ভূমিকা**

**(Introduction to Operating Systems)**

**প্রফেসর চেস্টার রেবেরো**

**(Prof. Chester Rebeiro)**

**কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ**

**(Department of Computer Science and Engineering),**

**ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস**

**(Indian Institute of Technology, Madras)**

**সপ্তাহ – 06**

**বক্তৃতা – 23**

**আন্তঃ প্রসেস আদানপ্রদান**

**(Inter Process Communication)**

এই ভিডিওতে আমরা আন্তঃ প্রসেস কমিউনিকেশনটি দেখব। মূলত, যখন আমরা বড় application লিখি তখন প্রায়ই তাদেরকে পৃথক প্রসেস হিসাবে লিখলে বেশি উপযোগী হয়। তাই এই প্রসেসগুলির মধ্যে একটি কার্যকর যোগাযোগ করার জন্য, inter process communication ব্যবহার করা হয়। এই ভিডিওটিতে, আমরা inter process communication বা IPC এর সংক্ষিপ্ত পরিচয় এবং কিছু উদাহরণ দেখব।

(স্লাইড সময় : 00:50 দেখুন)

আমরা প্রসেসের virtual memory view দেখেছি। আমরা বলেছি যে যখন একটি প্রসেস এক্সিকিউট করে তখন সেটি virtual memory address পাঠায় যা virtual memory map এ map করা হয়। আমরা দেখেছি যে একটি MMU রয়েছে যা memory তে রাখা page table ব্যবহার করে এবং এই virtual address গুলিকে physical address এ রূপান্তরিত করে। আমরা এও দেখেছি যে প্রসেসগুলি CPU তে execute করে তাদের প্রত্যেকের নিজস্ব virtual memory map থাকে।

(স্লাইড সময় : 1:23 দেখুন)

Virtual memory map এর ব্যবহার কিছু স্তরে abstraction (level of abstraction) প্রদান করে। মূলত, এক্সিকিউটিং প্রসেসটিই শুধুমাত্র তার virtual address গুলি জানতে পারবে, এবং এই virtual address গুলির সাহায্যে virtual memory map এর user space এ প্রবেশ করতে পারবে। আমরা দেখেছি যে MMU তারপর এই mapping কে সংশ্লিষ্ট physical address এ রূপান্তরিত করবে। সুতরাং, এক্সিকিউটিং প্রসেসটির কোনোমতেই জানতে পারে না যে তার memory access এর জন্য সংশ্লিষ্ট কোন physical address আছে। যেমন আপনি যদি একটি integer i আপনার প্রোগ্রামে declare করেন তবে আপনি printf ampersand i এর সাহায্যে i এর virtual address প্রিন্ট করতে পারেন। কিন্তু আপনার পক্ষে user space থেকে ওই variable i এর সংশ্লিষ্ট physical address নির্ধারণ করা সম্ভব নয়। দ্বিতীয় ব্যাপারটি হল এক্সিকিউটিং প্রসেসটি শুধুমাত্র নিজস্ব virtual memory map এ প্রবেশ করতে পারে। যদি অন্য কোন প্রসেস তার নিজের table এ থাকে, তবে user space থেকে এক্সিকিউটিং প্রসেসটি অন্য প্রসেসটির সম্পর্কে কিছু জানতে পারে না।

(স্লাইড সময় : 02:42 দেখুন)

সুতরাং, যখন একটি context switch হয় এবং একটি নতুন প্রসেস এক্সিকিউট করা হয় তখন দ্বিতীয় প্রসেসটির virtual memory map বা virtual memory address ব্যবহার করা হয়। তাই এই প্রসেসটির পক্ষে অন্য কোন প্রসেসের virtual memory address নির্ধারণ করার কোন উপায় নেই। এই দুটি স্লাইড থেকে আমরা দেখলাম যে একটি প্রসেসের পক্ষে অন্য কোন প্রসেসের কোন তথ্য জানার কোন উপায় নেই। এই অবস্থায় কিভাবে একটি প্রসেস অন্য প্রসেসের সঙ্গে যোগাযোগ করবে?

(স্লাইড সময় 03:15 দেখুন)

এটি করার জন্য, একটি কৌশল আছে যাকে inter process communication বলা হয়। মূলত IPC বা inter process communication এর মাধ্যমে, দুইটি প্রসেস নিজেদের মধ্যে তথ্য আদানপ্রদান করতে সক্ষম হয়। IPC এর সুবিধা হল যে প্রসেসগুলি modular হতে পারে আর return করতে পারে। তাই প্রতিটি প্রসেস একটি মাত্র কাজ করে এবং প্রসেসগুলি তারপর IPC এর মাধ্যমে একে অপরের সাথে যোগাযোগ করতে পারে। উদাহরণস্বরূপ, মনে করা যাক একটি data acquisition system এবং একটি control system আছে। আমরা এমন একটি প্রসেস লিখতে পারি যা বহির্জগৎ থেকে তথ্য, যেমন তাপমাত্রা, চাপ, বা গতি ইত্যাদি অর্জন করে, তারপর এটি এই সংগৃহীত তথ্যগুলি দ্বিতীয় প্রসেসকে পাঠায় যেটি এই তথ্য বিশ্লেষণ করে নির্দিষ্ট কিছু parameter নির্ণয় করে। এই parameter গুলি তৃতীয় একটি প্রসেসে পাঠানো যেতে পারে যা কিছু বাইরের তথ্যকে সক্রিয় করে, যেমন এটি একটি valve খুলতে বা ভালভ বন্ধ করতে পারে বা কোন ঘরের তাপমাত্রা নিয়ন্ত্রণ করতে পারে, ইত্যাদি। সুতরাং,

আমরা দেখছি যে আমরা আমাদের অ্যাপ্লিকেশনে একটি modular গঠন (structure) দিতে পারি। প্রতিটি প্রসেস শুধুমাত্র একটি দিকের প্রতি মনোসংযোগ করতে পারে। প্রসেসগুলির মধ্যে যোগাযোগ inter process communication এর মাধ্যমে অর্জিত হয়। সাধারণত অপারেটিং সিস্টেমে তিনটি উপায়ে IPC বাস্তবায়িত হয়। এইগুলি হল shared memory, message passing এবং সংকেত (signal) এর সাহায্যে মাধ্যমে হয়। এইগুলির প্রতিটির দিকে দেখা যাক।

(স্লাইড সময় 05:00 দেখুন)

Shared memory পদ্ধতিতে একটি প্রসেস RAM এ একটি ক্ষেত্র তৈরি করে যা অন্য একটি প্রসেসে ব্যবহৃত হয়। তাই মূলত, প্রসেস 1 এবং প্রসেস 2 এর মধ্যে যোগাযোগ এই বিশেষ shared memory এর মাধ্যমে হয়। উভয় প্রসেস এই shared memory কে সাধারণ কার্যকারী মেমরির মত ব্যবহার করতে পারে, তাই তারা একে অন্যের থেকে স্বাধীনভাবে এই নির্দিষ্ট মেমরিটিতে পড়তে বা লিখতে পারে। এর সুবিধা হল যে কোন system call হয় না, ফলে যোগাযোগ অত্যন্ত দ্রুত হয়। এখানে একমাত্র প্রয়োজন হল যে আপনি এখানে একটি array সংজ্ঞায়িত (define) করতে পারেন এবং তারপর shared memory তে এই array পূরণ করতে পারেন যা অন্য প্রসেস পড়তে পারবে। Shared memory পদ্ধতির সীমাবদ্ধতা হলো এটিতে ত্রুটি হওয়ার প্রবণতা থেকে যায়। এই কারণে দুটি প্রসেসের মধ্যে সমন্বয় করা প্রয়োজন। একটি ছোট উদাহরণ দ্বারা আমরা দেখব যে লিনাক্সে কিভাবে shared memory পদ্ধতিটি প্রয়োগ হয়।

(স্লাইড সময় 06:05 দেখুন)

লিনাক্স অপারেটিং সিস্টেমের shared memory তে মূলত তিনটি সিস্টেম কল ব্যবহার করা হয়। প্রথমটি হল 'shmget' বা shared memory get, যা তিনটি parameter নেয়; যথা key, size এবং flags। এই সিস্টেম কল একটি shared memory অংশ (segment) তৈরি করে। এটি return করে shared memory এর একটি ID যাকে 'shmid' বা shared memory ID বলা হয়। Parameter key হল shared memory segment এর unique identifier এবং size হল shared memory এর সাইজ। এটিকে সাধারণত 4 কিলোবাইট সাইজের page size এ rounding করা হয়। এইভাবে একটি প্রসেসের shared memory সৃষ্টি করা হয়। এখন অন্য একটি প্রসেস এই সের্ভার্ড মেমরির সঙ্গে সংযুক্ত হতে পারে 'shmat' বা shared memory attach এই নির্দিষ্ট সিস্টেম কল দ্বারা। এই বিশেষ সিস্টেম কলের জন্য shared memory ID, তার address এবং flags এর প্রয়োজন। এই সিস্টেম কলটি shared memory এর সাথে নতুন (calling) প্রসেসটির address space কে সংযুক্ত করে। Address টি হল shared memory address space এর একটি পয়েন্টার। আমরা একটি উদাহরণের মাধ্যমে এটি আরও বুঝতে পারব। Shared memory attach এর বিপরীত হল shared memory detach যার দ্বারা একটি প্রসেস shared

memory থেকে নিজের user space কে আলাদা করতে পারে। এই detach সিস্টেম কলের জন্য shared memory ID এর প্রয়োজন।

(স্লাইড সময় 07:40 দেখুন)

Shared memory এর একটি উদাহরণ দেখা যাক। ধরা যাক আমরা server dot c বলে একটি প্রোগ্রাম লিখেছি আর সেটির shared memory সৃষ্টি করেছি। আমরা 5678 বলে একটি key সংজ্ঞায়িত করি, যদিও এই key ইচ্ছামত যা খুশি নেওয়া যায় তবু এটি যেন shared memory কে অদ্বিতীয় ভাবে (uniquely) সনাক্ত করতে পারে। আমরা এই key এর মান এবং যে size এর shared memory সৃষ্টি করতে চাই সেই size বা shmsize shmget নামক function এ pass করি। মনে রাখতে হবে যে size যদিও আমরা শুধুমাত্র 27 byte এ নির্দিষ্ট করি, সেটি 4 KB অবধি বিস্তৃত page তৈরী করতে পারি। তৃতীয় parameter টি হল permission, এটি IPC\_CREAT হিসেবে দেওয়া হয় যা shared memory তে read, write ও execute এর permission হতে পারে। এবং অবশ্যই, যদি এই function ব্যর্থ হয় তাহলে এটি এখানে প্রবেশ করে আবার প্রশ্ন করে। অন্যথায়, এটি সফল হলে বৈধ shared memory ID পাওয়া যায়। পরবর্তী অংশে shared memory attach – shmat বলে ডাকা হয় যা এই নির্দিষ্ট ID প্রদান করে এবং আমরা এই নির্দিষ্ট shared memory এর একটি pointer পাই। যেমন char star shm এই shared memory এর pointer হতে পারে। এখন shared memory attach এই নির্দিষ্ট shared memory এর pointer return করে। এখন যদি এটি minus 1 return করে তার অর্থ কোন error হয়েছে এবং আমরা প্রশ্ন করছি। এখন এই নির্দিষ্ট সময়ে আমরা shared memory এর একটি pointer পেয়েছি যা C program এর pointer এর মত ব্যবহার করা যায়। উদাহরণস্বরূপ, এখানে pointer টিকে s variable এর উপর রেখেছি যেটি char star s বলে সংজ্ঞায়িত এবং আমরা এতে a থেকে z অক্ষরগুলি রেখেছি। এবং অবশেষে, আমরা একটি null termination দিয়ে এটি শেষ করেছি এবং তারপর ক্রমাগত নিষ্ক্রিয় অবস্থায় আমরা একটি loop এর মধ্যে দিয়ে যাচ্ছি, আমরা পরে এই দুইটি statement এ আসব। এবার এই code এর client এর দিকে দেখা যাক। আমরা এই shmget আহ্বান করব এবং এটিকে একটি key পাঠাব যা সেই 5678। shared memory এর size সেই 27 এবং আমরা প্রয়োজনীয় permission গুলি দেব। মনে রাখবেন যে আমাদের এখানে CREAT দিতে হবে না কারণ shared memory region অঞ্চল ইতিমধ্যেই তৈরি করা হয়েছে। এবার আমরা আগে মতই shared memory টিকে সংযুক্ত (attach) করব, shared memory attach এই 'shmat' function টিকে ডাকব এবং এটিতে shared memory ID টি পাঠিয়ে দেব। এবার আমরা একটি পয়েন্টার 'shm' পাব যা shared memory location এর pointer। সুতরাং, সার্ভার এবং ক্লায়েন্ট উভয়েই একই physical memory page কে নির্দেশ (point) করে। এখন আমরা এই বিশেষ shared memory ক্ষেত্র থেকে তথ্য পড়তে পারব। মনে রাখবেন যে server a থেকে z অর্থাৎ a, b, c, d থেকে z পর্যন্ত মান রেখেছে, এবং এ ক্ষেত্রে আমরা এই নির্দিষ্ট pointer s এর

দ্বারা এই a থেকে z এর মানগুলি পড়তে পারি এবং screen এ display করতে পারি। তাই shared memory থেকে সমস্ত তথ্য পড়া হলে এই star কে shared memory location এ পাঠানো হয়। এটি shared memory এর প্রথম location। মনে করে দেখুন server এ আমরা একটি while loop রেখেছি যেটি যতক্ষণ না সেটির প্রথম parameter টি shared memory কে নির্দেশ করে সেটি star হয় ততক্ষণ চলতে থাকে। সুতরাং যখন একটি star পাওয়া যায় তার অর্থ হল client এর সমস্ত তথ্য পড়া হয়েছে আর এবার server প্রস্থান (exit) করতে পারে। একই ভাবে client ও প্রস্থান করতে পারে। তাহলে দেখা যাচ্ছে যে server ও client দুইটি প্রসেস আছে। এই দুইটি প্রসেসের নিজেদের virtual address space রয়েছে, কিন্তু shared memory get এবং shared memory attach এর সাহায্যে যে shared memory region সৃষ্টি করা হল যা server এবং client উভয়ই ব্যবহার করতে পারে। এরপর আমরা shared memory region এর shm এ একটি pointer s পেয়েছি যেখানে server তথ্য দিতে পারে এবং client সেটি পড়তে পারে, এবং বিপরীতটিও সম্ভব। অবশেষে আমরা বলতে পারি যে server ও client এর মধ্যে সমন্বয়সাধন (synchronization) প্রয়োজন। এই synchronization টি client server module এ সুনির্দিষ্ট ভাবে program করা প্রয়োজন।

(স্লাইড সময় 12:54 দেখুন)

এবার আমরা বার্তা প্রেরণ বা message passing দেখব। Shared memory তে যেমন আমরা দেখলাম shared memory টি user space এর অংশ হিসেবে তৈরি হয়, message passing এ shared memory তৈরি হয় kernel এ। মূলত এখানে আমাদের send এবং receive এই system call গুলির প্রয়োজন হবে যাতে প্রসেসগুলি নিজেদের মধ্যে বার্তা আদান-প্রদান করতে পারে। যদি প্রসেস 2 প্রসেস 1 কে তথ্য পাঠাতে চায় তাহলে সেটি send system call করবে। এর ফলে kernel এক্সিকিউট করবে এবং shared memory তে তথ্য প্রেরিত হবে। অপরদিকে যদি প্রসেস 1 receive system call করে তাহলে সেটি shared memory থেকে তথ্য পড়তে পারবে। এই message passing এর সুবিধা হল এই যে এটি খুব সুস্পষ্ট। প্রসেস 1 ও প্রসেস 2 উভয়কেই নিজেদের মধ্যে তথ্য আদান-প্রদানের জন্য kernel এর সাহায্য নিতে হবে। এই পদ্ধতির সীমাবদ্ধতা হল যে এটি ধীর গতিতে হয় কারণ প্রতিটি send বা receive এর জন্য বার্তা একত্র করতে (marshalling) এবং ভাঙতে (demarshalling) হয়। আমরা জানি যে সাধারণভাবে একটি সিস্টেম কল যথেষ্ট খরচাসাপেক্ষ। সুতরাং, shared memory এর তুলনায় message passing বেশ ধীর গতির। যাইহোক, এটি shared memory এর তুলনায় কম ত্রুটিপ্ৰবণ কারণ sharing এর কাজটি kernel দ্বারা পরিচালিত হয় এবং kernel প্রসেস 1 এবং প্রসেস 2 এর মধ্যে সমন্বয় সাধন করে।

(স্লাইড সময় 14:27 দেখুন)

Message passing এর অন্য একটি প্রচলিত পদ্ধতি হল পাইপ (pipe) এর ব্যবহার। Pipe শুধুমাত্র parent ও child প্রসেসের মধ্যে ব্যবহার হয়। Parent থেকে child বা তদ্বিপরীত ভাবে pipe ব্যবহৃত হয়। Pipe এর অপর একটি ব্যাপার হল এটি একমুখী (unidirectional)। যখন একটি pipe সৃষ্টি করা হয় তখন তার সাথে সাধারণত দুইটি file descriptor এর সাথে সংযুক্ত করা হয়, fd[0] যেটি pipe থেকে পড়তে লাগে আর fd[1] যেটি pipe এ লিখতে লাগে। এইভাবে আমরা এই IPC এর একমুখী প্রকৃতিটি বুঝতে পারি। fd[১] শুধুমাত্র pipe buffer এ লিখতে কাজে লাগে আর fd[০] pipe buffer থেকে পড়তে কাজে লাগে।

(স্লাইড সময় 15:14 দেখুন)

Parent এবং child এর মধ্যে উভমুখী যোগাযোগ (bidirectional communication) এর জন্য pipe 0 এবং pipe 1, দুইটি pipe এর প্রয়োজন। যখন pipe 0 খোলা হবে তখন দুইটি file descriptor লাগবে, একটি pipe 0 তে লিখতে এবং অপরটি pipe 0 থেকে পড়তে। অনুরূপে pipe 1 এ লেখার জন্য এবং সেটি থেকে পড়ার জন্য দুইটি file descriptor লাগবে। একই ভাবে child process এর জন্য দুইটি file descriptor লাগবে। আসলে এই অতিরিক্ত file descriptor গুলির কোন প্রয়োজন নেই, প্রকৃতপক্ষে এই অতিরিক্ত file descriptor গুলি বন্ধ করেও এই ধরনের কোন কিছু পাওয়া যেতে পারে। Parent যদি child কে কোন তথ্য পাঠাতে চায় তাহলে parent pipe 0 তে লিখবে এবং child pipe 0 থেকে পড়বে। আবার child যদি parent কে কোন তথ্য পাঠাতে চায় তাহলে child pipe 1 এ লিখবে আর parent pipe 1 থেকে পড়বে।

(স্লাইড সময় 16:23 দেখুন)

আসুন User pipe এর একটি উদাহরণ দেখা যাক। এটি একটি আদর্শ লিনাক্স উদাহরণ এবং যেমন আমরা বলেছি, আমরা একটি parent এবং child প্রসেস সৃষ্টি করছি যেখানে parent থেকে child এ একটি unidirectional communication হবে। এবার এটি দেখা যাক। আমরা প্রথমে pipe নামক system call টি ডাকব এবং সেটি কে pipefd দেব। এখানে দুইটি element নিয়ে pipefd এর সংজ্ঞা দেওয়া হয় এবং fork বলে system call টিকে ডাকা হয়। আমরা পড়েছি যে fork system call এর ফলে parent এর হুবুহু সদৃশ একটি child প্রসেস সৃষ্টি করবে। Parent এ যে pid পাওয়া যায় সেটি হল child প্রসেসের pid যা fork return করে এবং এর মান 0 অপেক্ষা বেশি। অপরদিকে child প্রসেসের fork 0 return করে। Child প্রসেসে কি হয় সেটি প্রথমে দেখা যাক। Child প্রসেসে যেহেতু pid এর মান 0 সেহেতু এটি এখানে প্রথমে আসবে এবং আমরা যেটি প্রথম করব তা হল pipefd[0] কে বন্ধ করে দেব। pipefd[0] কে বন্ধ করা মানে হল আমরা child এর read pipe টি বন্ধ করে দিচ্ছি। অর্থাৎ আমাদের এখানে এখন pipe 0 এর সংশ্লিষ্ট দুইটি pipe রইল আর pipe 1 অর্থাৎ

ডানদিকের file descriptor টি খোলা রইল। এবার দেখা যাক child প্রসেসে কি হচ্ছে। Child প্রসেসে fork 0 return করেছে যার ফলে এই নির্দিষ্ট code টি এক্সিকিউট হবে। এই code টিতে আমরা প্রথমেই pipefd[0] কে বন্ধ করেছি। এর অর্থ হল read pipe এর সঙ্গে যুক্ত file descriptor টি বন্ধ করা হয়েছে। দ্বিতীয়ত আমরা file descriptor 1 এর সংশ্লিষ্ট fdopen বলে দ্বিতীয় একটি file open করছি যাকে write mode খোলা হচ্ছে এবং এবার আমরা fprintf out “Hello World” লিখতে পারি। মূলত আমরা যা pipe 1 এ “Hello World” লিখছি। যেহেতু fork এর return value 0 এর থেকে বেশি, parent প্রসেসটি চলবে, আমরা write pipe টি বন্ধ করে fdopen দ্বারা read pipe টি খুলব। এবার আমরা fscanf ব্যবহার করে pipe এ যা লেখা হয়েছে, অর্থাৎ “Hello World” সেটি পড়ব এবং স্ক্রীনে লিখব। আমরা যেটা ব্যবস্থা করলাম তা এই নীচের অংশে। সুতরাং দেখা গেল যে child প্রসেস pipe টি write mode এ খুলে তাতে “Hello World” লিখল আর parent প্রসেস pipe 1 টি read mode এ খুলে সেটি থেকে “Hello World” পড়ল। এইভাবে “Hello World” string টি child থেকে parent প্রসেসে স্থানান্তরিত হল। আপনি এই বিশেষ program টি লিনাক্স সিস্টেমে লিখে চালিয়ে দেখতে পারেন।

(স্লাইড সময় 19:58 দেখুন)

Message passing, shared memory এবং pipe ছাড়াও সঙ্কেতের (signals) মাধ্যমে inter process communication হতে পারে। এই signals হল প্রসেসের মধ্যে অসমন্বিত একমুখী যোগাযোগ (asynchronous unidirectional communication) এর মাধ্যম। অপারেটিং সিস্টেমে কিছু পূর্বসংজ্ঞায়িত signals এর সংজ্ঞা দেয় যা অপারেটিং সিস্টেম থেকে প্রসেস বা প্রসেস থেকে প্রসেস পার্থক্যে যেতে পারে। Signal গুলি হল ছোট পূর্ণসংখ্যা (small integer) যার প্রতিটির একটি করে পূর্বনির্ধারিত অর্থ আছে, যেমন 9 মানে একটি প্রসেসকে kill করা বা 11 মানে segmentation fault ইত্যাদি। প্রসেসকে কোন signal পাঠাতে যেমন kill এই pid তে signal পাঠানো হয় এবং আমরা integer সিগনাল নাম্বারটিও পাঠাতে পারি। প্রসেসটি এই signal handle করার জন্য signal বলে একটি function ব্যবহার করে যেটি signal এর এই 9 বা 11 নম্বরটি নেয় আর তারপর handler। Handler হল এই বিশেষ নাম্বারের জন্য signal handler এর একটি pointer। তাই ফলস্বরূপ আমরা একটি প্রক্রিয়া থেকে অন্য একটিতে signal পাঠাতে পারি এবং signal এর ধরনের উপর নির্ভর করে সংশ্লিষ্ট হ্যান্ডলার সম্পাদন করা হবে। এই ভিডিওতে আমরা IPC এর একটি সংক্ষিপ্ত ভূমিকা দেখেছি। মূলত, আমরা বিভিন্ন ধরনের IPC দেখেছি যেমন, message passing, shared memory, pipe এবং signal এর মাধ্যমে asynchronous transfer। প্রসেসগুলির মধ্যে যোগাযোগের জন্য এই IPC ব্যাপকভাবে ব্যবহার করা হয়। এবং তাই, আমরা সম্পূর্ণ প্রোগ্রামের মধ্যে শুধুমাত্র একটি নির্দিষ্ট দৃষ্টিভঙ্গির উপর মনোযোগ নিবদ্ধ করে প্রতিটি প্রোগ্রামের সঙ্গে, ছোট অত্যন্ত modular অ্যাপ্লিকেশন করতে সক্ষম। ধন্যবাদ।